

78. Compter ce que coûte une boucle

Le compilateur ne cache pas ses prix.

1. Ce qu'il faut comprendre

Compiler donne son compte rendu : octets de programme, fonctions, variables, mémoire de travail. **Changer une ligne et recompiler montre ce qu'elle a coûté** — c'est la seule mesure qui ne mente pas.

Deux choses à savoir, mesurées et non supposées. `100 * x` et `x * 100` ne coûtent pas la même chose : la routine générale boucle sur son opérande de droite, alors le compilateur met le nombre connu à droite quand l'opérateur s'en moque.

Et `x / 16` devient quatre décalages, `x % 16` un `and` — **quand le diviseur est connu**. Sinon, c'est la routine générale, et elle se paie.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1361 octets** de cartouche.

1. UNE TABLE GRAVÉE DANS LA CARTOUCHE — TABLE

```
const uint8_t TABLE[] = { 3, 1, 4, 1, 5, 9, 2, 6 };
```

2. UNE FONCTION — TOTAL()

```
uint8_t total() {  
    uint8_t somme = 0;  
    for (uint8_t i = 0; i < sizeof(TABLE); i++) somme += TABLE[i];  
    return somme;  
}
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

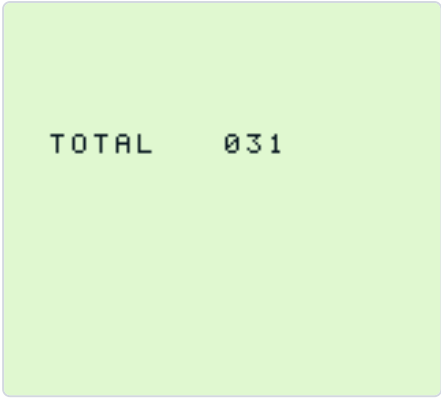
```
int main() {  
    texte(2, 6, "TOTAL");  
    nombre(10, 6, total());  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

031 — la somme de la table gravée.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la somme est juste
- ✓ l'étiquette est là